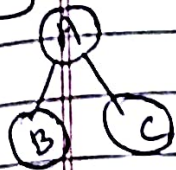


Tree and binary search Trees.

Tree = a collection of nodes connected by edges, with one special node called root, where every other node has exactly one parent and there are no cycles.

A tree with n nodes has exactly $n-1$ edges.

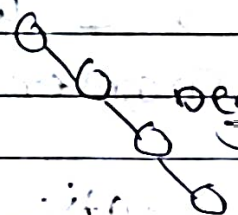
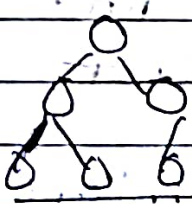
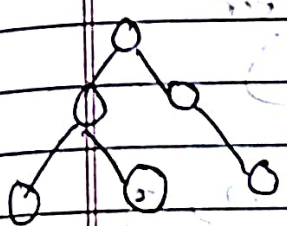
e.g. family tree, an HTML page.



A is root and b, c is child of A

height of = edges

edges: 3, which means height 3



degenerate

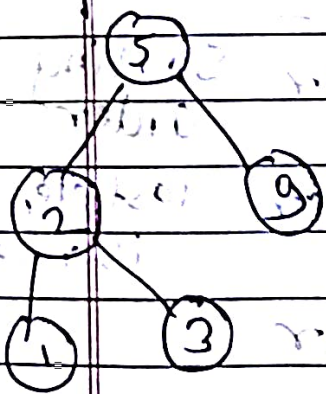
grow only one side.

Balanced... complete

leaf node is

left to right

Binary search tree



left is smaller than node
node is smaller right.

Search in $O(h)$

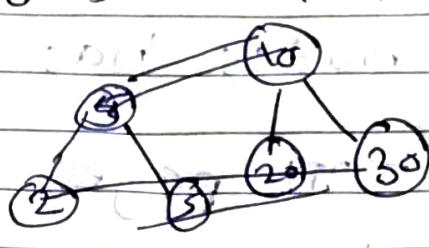
left < ~~right~~ < right
node

Double link list

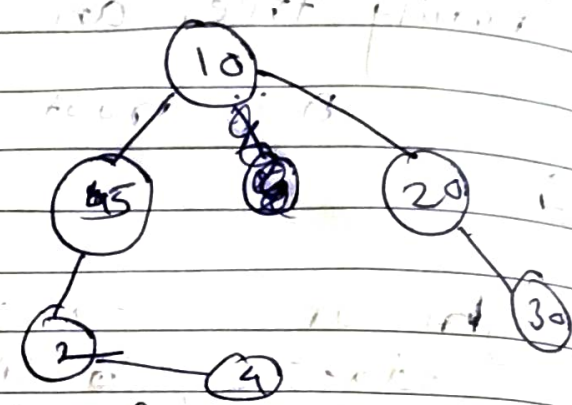
Tree traversal

DFS: pre order, ~~in~~ post order
~~DFS~~ Root left right

10 5 2 4 20 30



large element 20, 30
 small = 5, 2, 4



10 5 2 4 20 30 find inorder
 => Inorder (left Root Right)

```

static void inorder(Node n) {
    if (n == null) return;
    inorder(n.left);
    middle
    System.out.println(n.data);
    inorder(n.right);
}
    
```

2, 4, 5, 10, 20, 30 inorder Sorting order

4 2, 5, 30, 20, 10 Postorder last element is root

10, 5, 2 4, 20 30 Preorder

~~in postorder~~

```

static void preorder(Node n)
{
    if (n == null) return;
    System.out.println(n.data + " ");
    preorder(n.left);
    preorder(n.right);
}
    
```

```

static void postorder(Node n)
{
    if (n == null) return;
    postorder(n.left);
    postorder(n.right);
    System.out.println(n.data);
}
    
```

static void inorder(Node n)

⇒ This is static method inorder. It takes n as an argument.

if (n == null) return;
 check whether current node is null.
 If it is not, return from method

inorder(n.left);
 recursively call inorder from the left child.

I recursively call inorder from.
 System.out.println(n.data)
 print data of the current node.

Tree and terminology

Tree Traversals.

complete, complete, Almost complete
complete binary tree.

Array implementation of A, B, T
Binary Search trees
AVL Tree.

* Trees help in efficient data organization and retrieval for hierarchical data.

Trees useful for storing data that naturally forms hierarchy.

e.g. file systems on computers are structured as trees. With folders containing subfolders and files.

Parent node

* Types of Tree data Structures.

Tree data can be classified based on number of children each node can have.

binary tree:- each node can have maximum of 2 children.

1) full binary tree = every node has 0 either 2 children.

2) complete tree = all levels are fully filled, which is filled from left to right.

3) Balanced binary tree = height difference between left and right subtree of every node is minimal.

* properties of Tree data structure.

1) * Number of edges: edge is connection between two nodes.

A tree with n nodes, will always $n-1$ edges.

2) * Depth of node: the depth of node is the length of path, number of edges from root to that node.

3) Height of tree:-

is ~~long~~ length of longest path from root to leaf node.

4) Degree of nodes is a number of subtrees attached to it. e.g. number of children it has.

formula

maximum nodes $2^{(h+1)} - 1$

e.g. $h=2$

$$2^3 - 1$$

$$= 7$$

maximum nodes at level n

2^n

e.g. level 3

$$2^3 = 8 \text{ node}$$

Tree complexity

	Binary Tree	BST	Worst	AVL
Search	$O(n)$	Average $O(\log n)$	$O(n)$	$O(\log n)$
Insert	$O(n)^*$	$O(\log n)$	$O(n)$	$O(\log n)$
Delete	$O(n)$	$O(\log)$	$O(n)$	$O(\log n)$

implementation

left child

$$2 * i + 1$$

parent (1) 11/1

eg value
index

Tree

index 1



for node 2

index 1

left $2(1) + 1 = 3$

right $2 * i + 2 =$

$$2(1) + 2 = 5$$

```

import java.util.*;
class Node {
    int data;
    Node left;
    Node right;
    Node(int data) {
        this.data = data;
        left = null;
        right = null;
    }
}

```

```

public class main {
    static Node root = null;
}

```

// create insert

```

static Node insert (Node root, int value) {
    if (root == null) {
        return new Node (value);
    }
}

```

```

    if (value < root.data) {

```

```

        root.left = insert (root.left, value);
    }

```

```

    else if (value > root.data) {

```

```

        root.right = insert (root.right, value);
    }

```

```

    else {

```

```

        System.out.println("Duplicate not allowed");
    }

```

```

    return root;
}

```

preorder.

```
public void  
static void preorder(Node root)  
{  
    if (root != null) {  
        System.out.println(root.data);  
        preorder(root.left);  
        preorder(root.right);  
    }  
}
```

Inorder

```
static void inorder(Node root)  
{  
    if (root != null) {  
        inorder(root.left);  
        System.out.println(root.data);  
        inorder(root.right);  
    }  
}
```

Postorder

```
static void postorder(Node root)  
{  
    if (root != null) {  
        postorder(root.left);  
        postorder(root.right);  
        System.out.println(root.data);  
    }  
}
```

BFS / Level order

```
static void BFS(Node root)
```

```
{  
    if (root == null)  
    {  
        System.out.println("Tree is empty");  
        return;  
    }
```

```
    Queue<Node> queue = new LinkedList<>();
```

```
    queue.add(root);
```

```
    while (!queue.isEmpty()) {
```

```
        Node current = queue.remove();
```

```
        System.out.println(current.data + " ");
```

```
        if (current.left != null) {
```

```
            queue.add(current.left);
```

```
        if (current.right != null) {
```

```
            queue.add(current.right);
```

```
    }
```

find minimum

```
static Node findmin(Node root)
{
    while (root != null)
    {
        root = root.left;
    }
    return root;
}
```

delete

```
static Node delete(Node root, int value)
{
    if (root == null)
    {
        value not found
        return null;
    }
```

```
    if (value < root.data)
    {
        root.left = delete(root.left, value);
    }
```

```
    else if (value > root.data)
    {
        root.right = delete(root.right, value);
    }
```

else {

```
    if (root.left == null && root.right == null)
    {
        return null;
    }
```

only
right
child

```
if (root.left == null) {
    return root.right;
}
```

only
left
child

```
if (root.right == null) {
    return root.left;
}
```

Two children =

```
node Successor = findmin(root.right);
root.data = Successor.data;
root.right = Successor;
delete(root.right, Successor.data);
```

```
return root;
}
```

1. "up" process) finding two nodes
2. "down" process) deleting two nodes
3. "up" process) finding two nodes

```
Static void searchkey(int value)
```

```
if (root == null)
```

```
value = root.data
```

```
root.right = searchkey(value)
```

```
Static boolean Search(Node root, int key)
```

```
if (root == null)
```

```
return false
```

```
if (root.data == key)
```

```
return true;
```

```
if (key < root.data)
```

```
return Search(root.left, key);
```

```
else
```

```
return Search(root.right, key);
```

```
main
```

```
System.out.println("Search go");
```

```
System.out.println(Search(root, go)?  
"found" : "Not found");
```